Lesson 2 Introducing the Warning Light Function

In this lesson, we will learn how to use PiCar-b's warning light function.

2.1 Function Overview

This tutorial is about how to use multi-threading to achieve some effects related to WS2812 LED lights. Multi-threading is common in robot projects, since robots have high requirements for real-time response. For each task performing, try not to block the main thread communication.

Multi-threading is similar to executing multiple different programs or tasks at the same time. Multi-threaded operation has the following advantages:

1. Using threads to put time-consuming tasks in the background for processing.

2. Improving the efficiency of the program. In the subsequent real-time video and OpenCV processing video frames, multi-threading is used to greatly increase the frame rate.

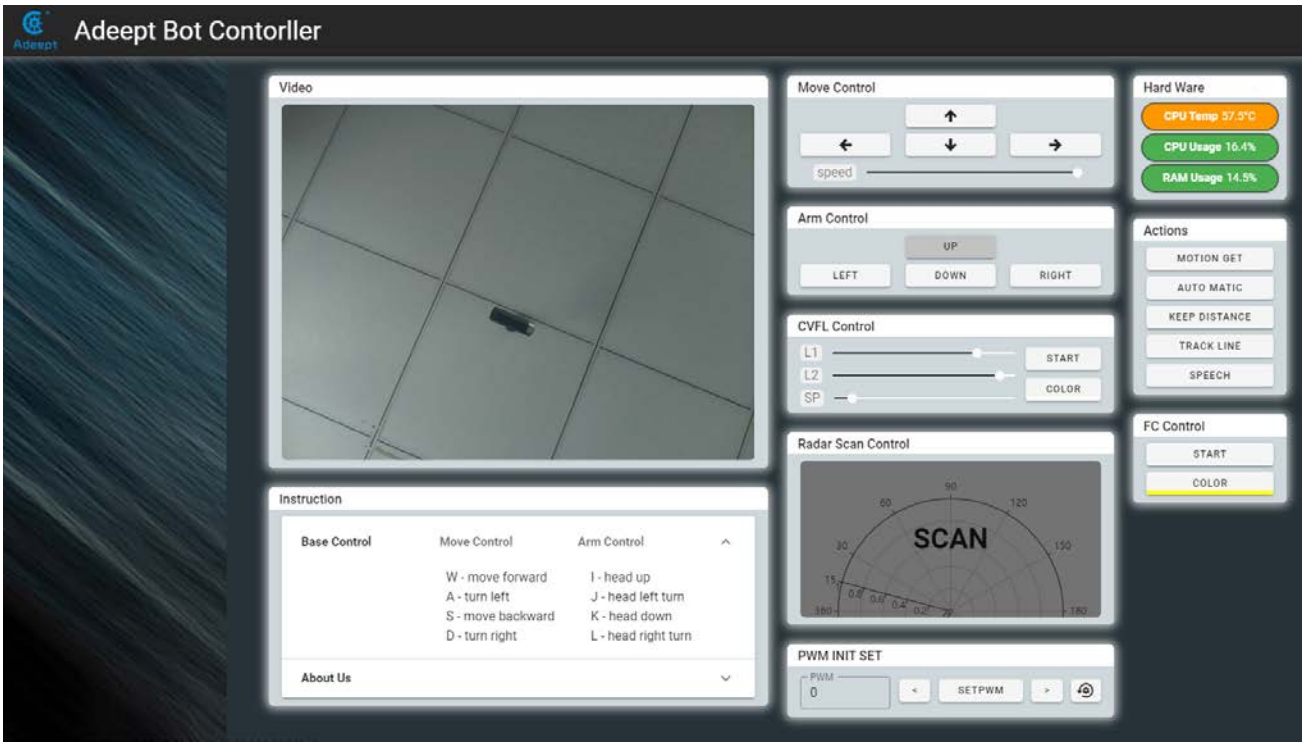
3. It's more convenient to call an encapsulated multi-threaded task, similar to the non-blocking control method – in other words, the control of the servo is encapsulated by multi-threading.

We use Python's threading library to provide thread-related work, and thread is the smallest unit of work in an application. For the current version of Python, there are no priorities, no thread groups, and threads cannot be stopped, suspended, resumed, or interrupted.

2.2 Running the Warning Light Program

1. Start the PiCar-b Robot. It may take about 30-50s to boot.

2. After PiCar-b is turned on, open the Chrome browser on your mobile or computer, enter the IP address of your Raspberry Pi and access port ":5000" into the IP address bar, like this: 192.168.3.44:5000. The web controller will then be displayed on the browser.



3. Click "POLICE LIGHT", and PiCar-b will flash lights of different colors.

4. Click "POLICE LIGHT" again to stop the function.

2.3 Main Program

For the complete code, please refer to the file robotLight.py.

```
import time
import sys
from rpi_ws281x import *
import threading


"""
Use the Threading module to create threads, inherit directly from threading.Thread, and then override the __init__ method and the run method
'''
```

```
class RobotLight(threading.Thread):
    def __init__(self, *args, **kwargs):
        """
        Here initialize some settings about LED lights
        '''
        self.LED_COUNT      = 16      # Number of LED pixels.
        self.LED_PIN        = 12      # GPIO pin connected to the pixels (18 uses PWM!).
        self.LED_FREQ_HZ    = 800000  # LED signal frequency in hertz (usually 800khz)
        self.LED_DMA        = 10      # DMA channel to use for generating signal (try 10)
        self.LED_BRIGHTNESS = 255     # Set to 0 for darkest and 255 for brightest
        self.LED_INVERT     = False   # True to invert the signal (when using NPN transistor level shift)
        self.LED_CHANNEL    = 0       # set to '1' for GPIOs 13, 19, 41, 45 or 53

        """
        Set the brightness of the three RGB color channels, no need to change here, these values will be automatically set after the subsequent call of the breathing light function
        '''
        self.colorBreathR = 0
        self.colorBreathG = 0
        self.colorBreathB = 0
        self.breathSteps = 10

        """
        The mode variable, 'none' will make the thread block and hang, the light will not change;
         'police' is a police light mode, red and blue flash alternately;
         'breath' breathing light, you can set the specified color.
        '''
        self.lightMode = 'none'     #'none' 'police' 'breath'

        # Create NeoPixel object with appropriate configuration.
        self.strip = Adafruit_NeoPixel(self.LED_COUNT, self.LED_PIN, self.LED_FREQ_HZ,
                                       self.LED_DMA, self.LED_INVERT, self.LED_BRIGHTNESS,
                                       self.LED_CHANNEL)
        # Intialize the library (must be called once before other functions).
        self.strip.begin()

        super(RobotLight, self).__init__(*args, **kwargs)
```

```
        self.__flag = threading.Event()
        self.__flag.clear()

    # Define functions which animate LEDs in various ways.
    def setColor(self, R, G, B):
        """
        Set the color of all lights
        """
        color = Color(int(R),int(G),int(B))
        for i in range(self.strip.numPixels()):
            self.strip.setPixelColor(i, color)
            self.strip.show()

    def setSomeColor(self, R, G, B, ID):
        """
        Set the color of some lamps, the ID is the array of the serial number of this lamp
        """
        color = Color(int(R),int(G),int(B))
        #print(int(R),'  ',int(G),'  ',int(B))
        for i in ID:
            self.strip.setPixelColor(i, color)
            self.strip.show()

    def pause(self):
        """
        Call this function, set __flag to False, block the thread
        """
        self.lightMode = 'none'
        self.setColor(0,0,0)
        self.__flag.clear()

    def resume(self):
        """
        Call this function, set __flag to True to start the thread
        """
        self.__flag.set()

    def police(self):
        """
        Call this function to turn on the police light mode
        """
        self.lightMode = 'police'
```

```
            self.resume()


    def policeProcessing(self):
        """
        The specific realization of the police light mode
        '''
        while self.lightMode == 'police':
            """
            Blue flashes 3 times
            '''
            for i in range(0,3):
                self.setSomeColor(0,0,255,[0,1,2,3,4,5,6,7,8,9,10,11])
                time.sleep(0.05)
                self.setSomeColor(0,0,0,[0,1,2,3,4,5,6,7,8,9,10,11])
                time.sleep(0.05)
            if self.lightMode != 'police':
                break
            time.sleep(0.1)
            """
            Red flashes 3 times
            '''
            for i in range(0,3):
                self.setSomeColor(255,0,0,[0,1,2,3,4,5,6,7,8,9,10,11])
                time.sleep(0.05)
                self.setSomeColor(0,0,0,[0,1,2,3,4,5,6,7,8,9,10,11])
                time.sleep(0.05)
            time.sleep(0.1)

    def breath(self, R_input, G_input, B_input):
        """
        Call this function to turn on the breathing light mode, you need to enter three parameters, namely the brightness of the RGB three color channels, as the color when the brightness of the breathing lamp is maximum
        '''
        self.lightMode = 'breath'
        self.colorBreathR = R_input
        self.colorBreathG = G_input
        self.colorBreathB = B_input
        self.resume()

    def breathProcessing(self):
```

```
        """
        Specific realization method of breathing lamp
        '''
        while self.lightMode == 'breath':
            """
            All lights gradually brighten
            '''
            for i in range(0,self.breathSteps):
                if self.lightMode != 'breath':
                    break
                self.setColor(self.colorBreathR*i/self.breathSteps,
                              self.colorBreathG*i/self.breathSteps,
                              self.colorBreathB*i/self.breathSteps)
                time.sleep(0.03)
            """
            All lights are getting darker
            '''
            for i in range(0,self.breathSteps):
                if self.lightMode != 'breath':
                    break
                self.setColor(self.colorBreathR-(self.colorBreathR*i/self.breathSteps),
                              self.colorBreathG-(self.colorBreathG*i/self.breathSteps),
                              self.colorBreathB-(self.colorBreathB*i/self.breathSteps))
                time.sleep(0.03)

    def lightChange(self):
        """
        This function is used to select the task to perform
        '''
        if self.lightMode == 'none':
            self.pause()
        elif self.lightMode == 'police':
            self.policeProcessing()
        elif self.lightMode == 'breath':
            self.breathProcessing()

    def run(self):
        """
        Functions for multi-threaded tasks
        '''
        while 1:
            self.__flag.wait()
```

```
                self.lightChange()
                pass

if __name__ == '__main__':
    RL=RobotLight() # Instantiate the object that controls the LED light
    RL.start()      # Start thread

    """
    Start breathing light mode and stop after 15 seconds
    '''
    RL.breath(70,70,255)
    time.sleep(15)
    RL.pause()

    """
    Pause for 2 seconds
    '''
    time.sleep(2)

    """
    Start the police light mode and stop after 15 seconds
    '''
    RL.police()
    time.sleep(15)
    RL.pause()
```

2.4 Warning Lights or Breathing Lights in Other Projects

When your project needs to use LED lights for warning lights or breathing lights, you don't need to completely rewrite the code above but just copy *robotLight.py* in the *server* folder in the robot's package to the same folder of your project, and use the following code to make a warning light or breathing light:

```
import robotLight

RL=robotLight.RobotLight()  # Instantiate the object that controls the LED light
RL.start()      # Start thread
```

```
'''''
Start breathing light mode and stop after 15 seconds
'''
RL.breath(70,70,255)
time.sleep(15)
RL.pause()

'''''
Pause for 2 seconds
'''
time.sleep(2)

'''''
Start the police light mode and stop after 15 seconds
'''
RL.police()
time.sleep(15)
RL.pause()
```